

```

import tensorflow as tf from tensorflow.keras.models import load_model from
tensorflow.keras.preprocessing import image import numpy as np from
tensorflow.keras.preprocessing.image import img_to_array import cv2 import matplotlib.pyplot
as plt from datetime import datetime, timedelta, time import os import yolov5 from threading
import Thread import torch import time from pygame import mixer import base64 import sqlite3
import datetime from flask import Flask, render_template, Response, jsonify import requests

```

```

app = Flask(name)

```

```

model1 = yolov5.load("Models/yolocff.pt") sclasses = model1.names device = 'cuda' if
torch.cuda.is_available() else 'cpu'

```

```

def score_frame(frame): model1.to(device) frame = [frame] results = model1(frame)
print(results) labels, cord = results.xyxy[0][:, -1], results.xyxy[0][:, :-1] return labels, cord

```

```

def class_to_label(x): return sclasses[int(x)]

```

```

fire_detected = False

```

```

def plot_boxes(results, frame): global fire_detected

```

```

labels, cord = results
n = len(labels)
print('n',n)
x_shape, y_shape = frame.shape[1], frame.shape[0]

```

```

for i in range(n):
    row = cord[i]
    if row[4] >= 0.2:
        x1, y1, x2, y2 = int(row[0] * x_shape), int(row[1] * y_shape), int(row[2] * x_shape),
int(row[3] * y_shape)
        bgr = (0, 0, 255)
        cv2.rectangle(frame, (x1, y1), (x2, y2), bgr, 2)
        cv2.putText(frame, class_to_label(labels[i]), (x1, y1), cv2.FONT_HERSHEY_SIMPLEX,
1, bgr, 4)

```

```

    if n == 1:
        mixer.init()
        sound = mixer.Sound('fire_alarm.ogg')
        sound.play()

```

```
return frame
```

```
def generate_frames(): camera = cv2.VideoCapture(0) while True: success, frame =  
camera.read() if not success: break else: results = score_frame(frame) frame =  
plot_boxes(results, frame) ret, buffer = cv2.imencode('.jpg', frame) frame = buffer.tobytes() yield  
(b'--frame\r\n' b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')
```

```
@app.route('/') @app.route('/index') def index(): return render_template('index.html')
```

```
@app.route('/camera') def camera(): return render_template('camera.html')
```

```
@app.route('/video_feed') def video_feed(): return Response(generate_frames(),  
mimetype='multipart/x-mixed-replace; boundary=frame')
```

```
if name == "main": app.run(port=600)
```